

Machine Learning Experiment no 4 - Logistic Regression

Libraries

```
In [2]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

```
In [3]: df1 = pd.read_csv('diabetes.csv')
```

```
In [5]: df1.head()
```

```
Out[5]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunc
0	6	148	72	35	0	33.6	C
1	1	85	66	29	0	26.6	C
2	8	183	64	0	0	23.3	C
3	1	89	66	23	94	28.1	C
4	0	137	40	35	168	43.1	2

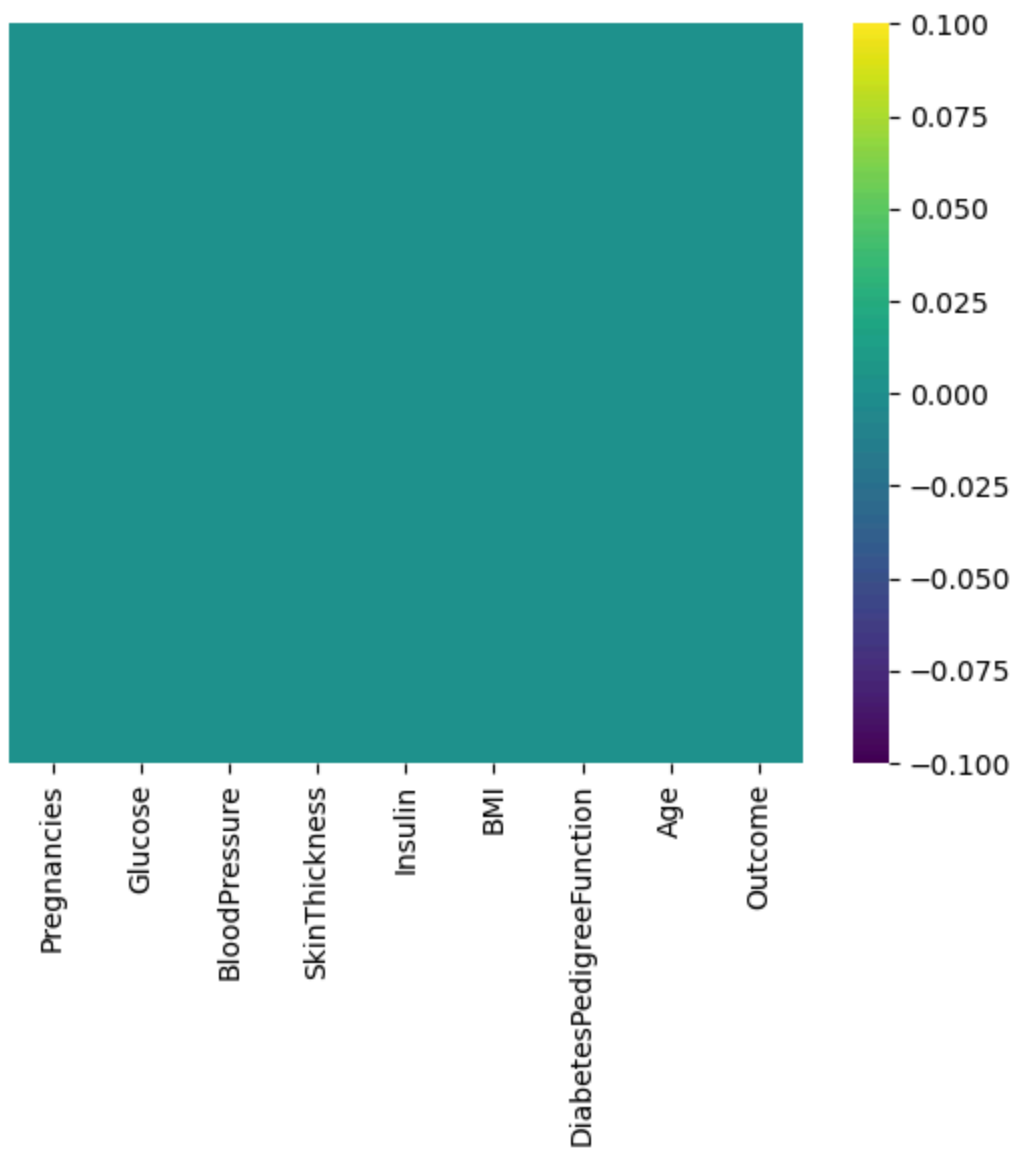
```
In [7]: df1.describe()
```

```
Out[7]:
```

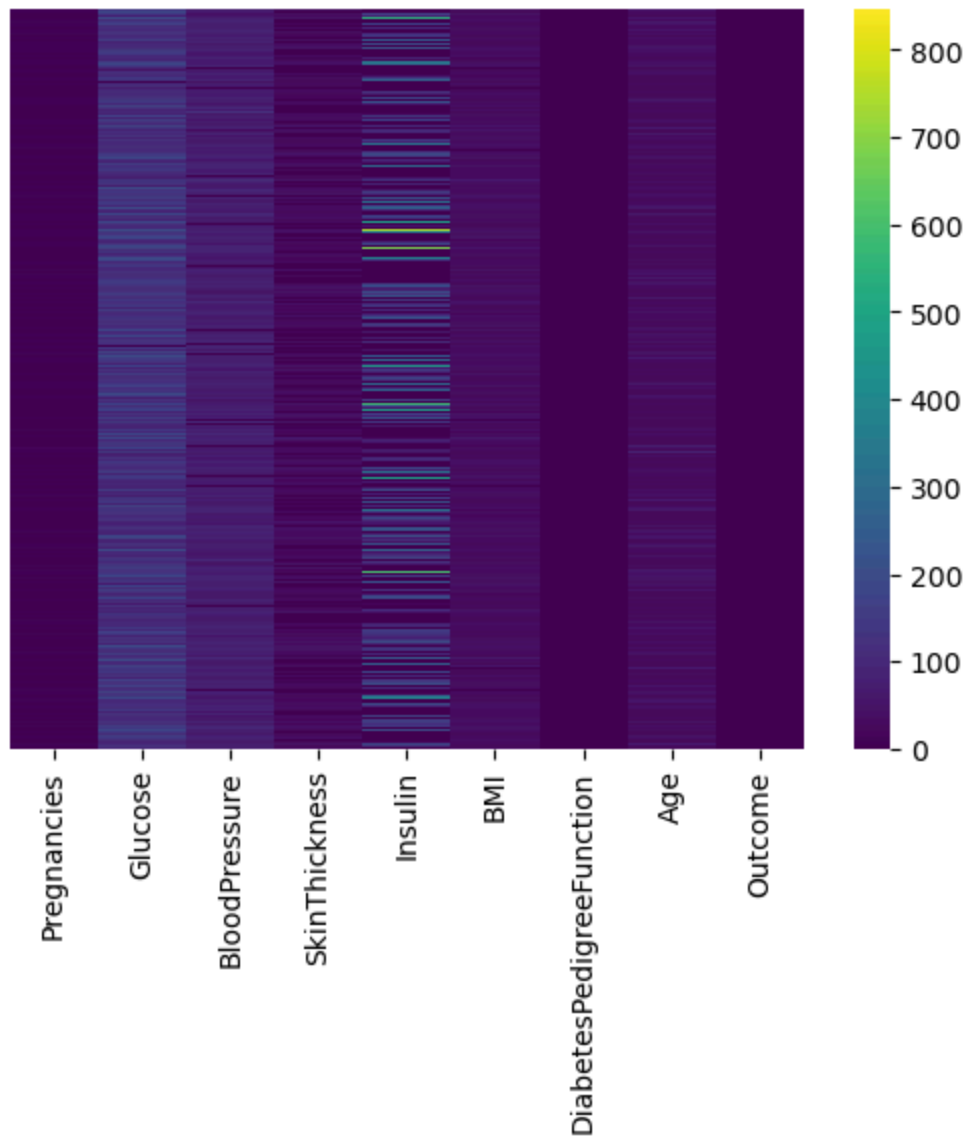
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Dia
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	

```
In [8]: sns.heatmap(df1.isnull(),yticklabels=False,cmap='viridis')
```

Out[8]: <Axes: >

In [9]: `sns.heatmap(df1,yticklabels=False,cmap='viridis')`

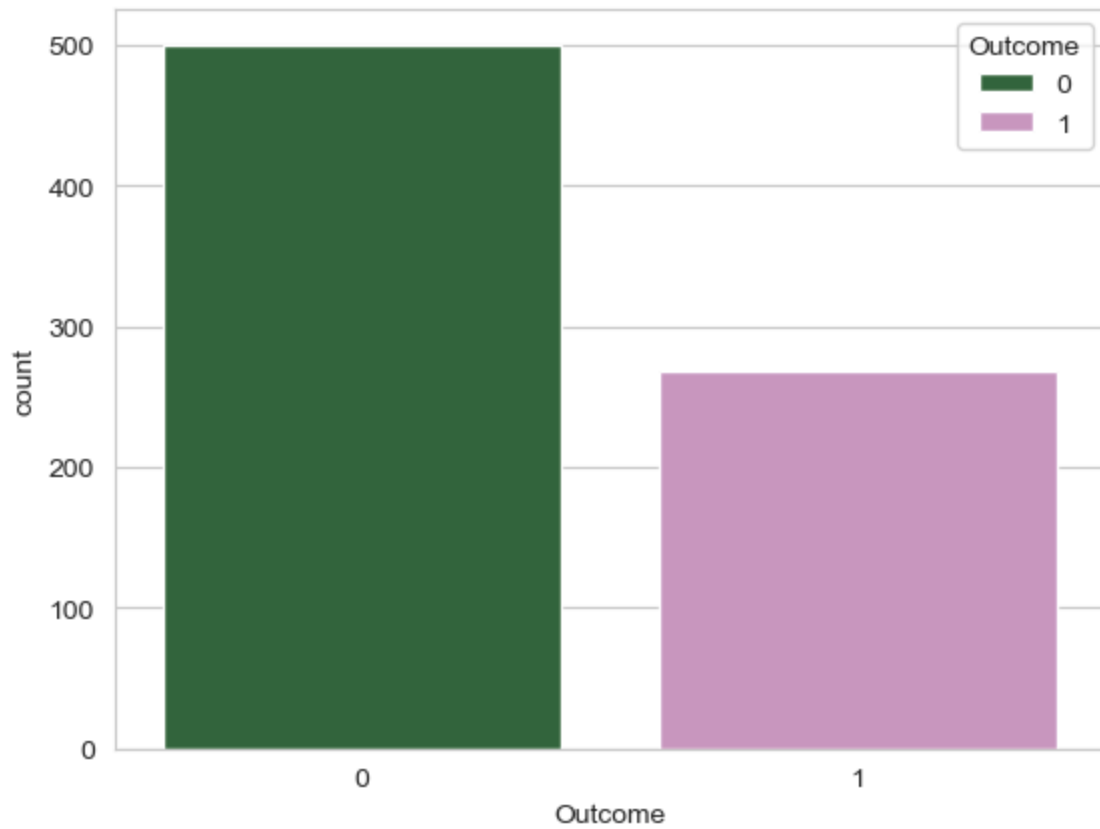
Out[9]: <Axes: >



```
In [10]: sns.set_style('whitegrid')
```

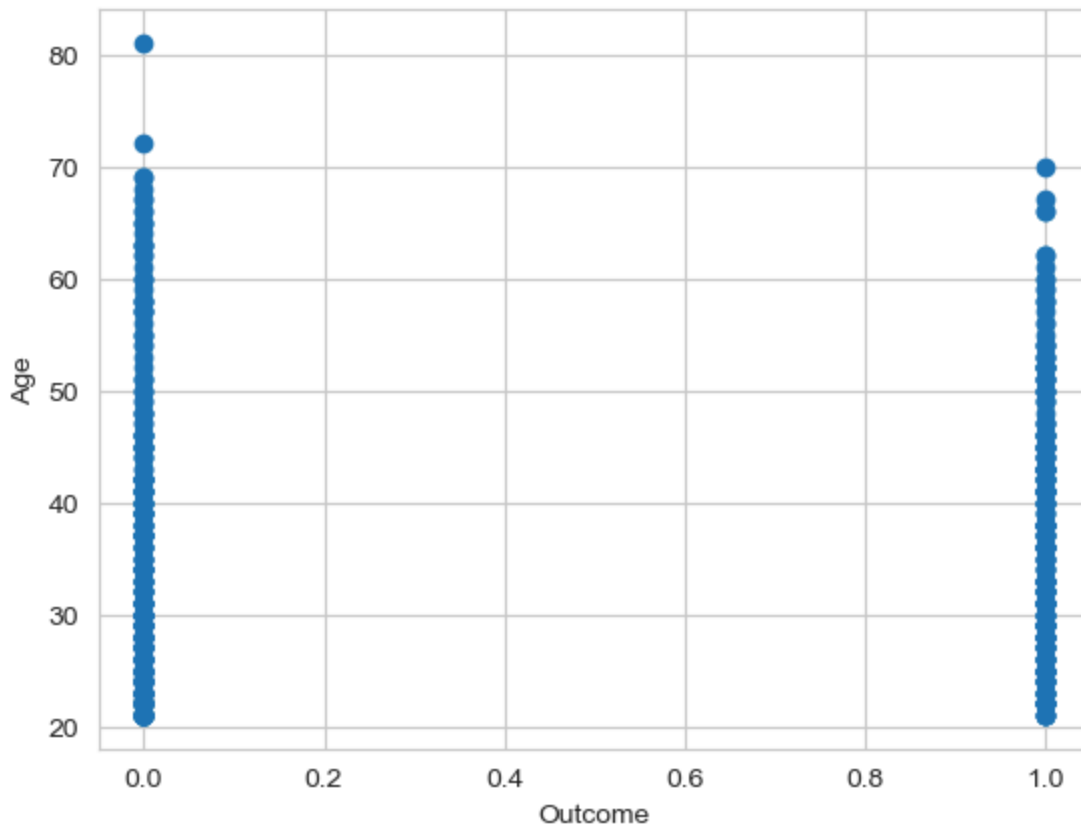
```
In [11]: sns.countplot(x='Outcome', hue='Outcome', data=df1, palette='cubehelix')
```

```
Out[11]: <Axes: xlabel='Outcome', ylabel='count'>
```



```
In [12]: plt.scatter(x='Outcome',y='Age',data=df1)
plt.ylabel('Age')
plt.xlabel('Outcome')
```

```
Out[12]: Text(0.5, 0, 'Outcome')
```



```
In [13]: sns.distplot(df1['Age'],kde=False,color='darkblue',bins=30)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_26436\2638141722.py:1: UserWarning:

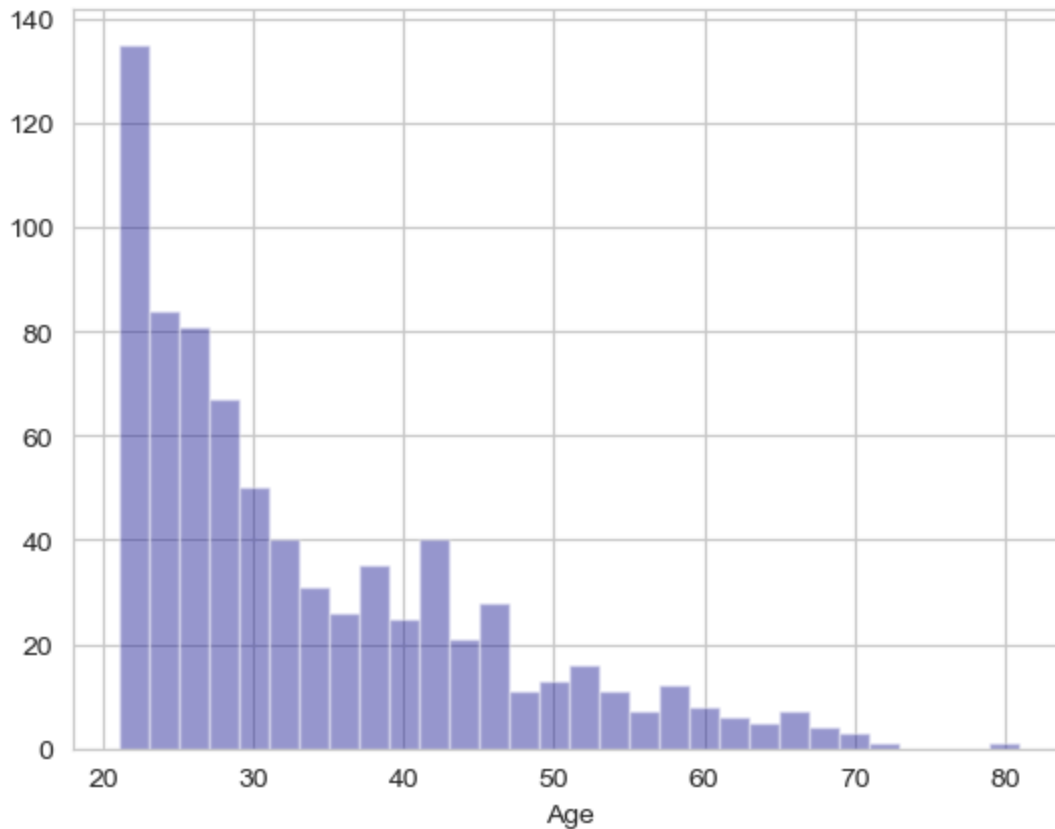
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(df1['Age'],kde=False,color='darkblue',bins=30)
```

```
Out[13]: <Axes: xlabel='Age'>
```



```
In [14]: sns.distplot(df1['BloodPressure'],kde=False,color='royalblue',bins=20)
```

C:\Users\HP\AppData\Local\Temp\ipykernel_26436\3905876826.py:1: UserWarning:

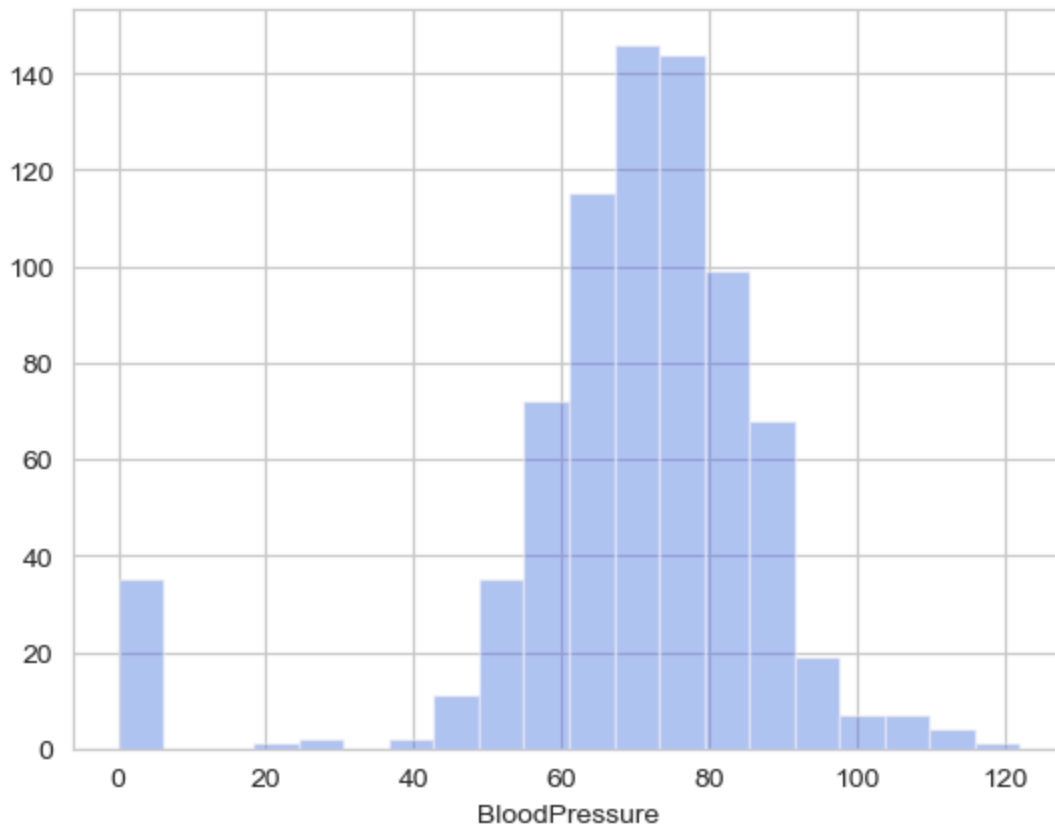
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

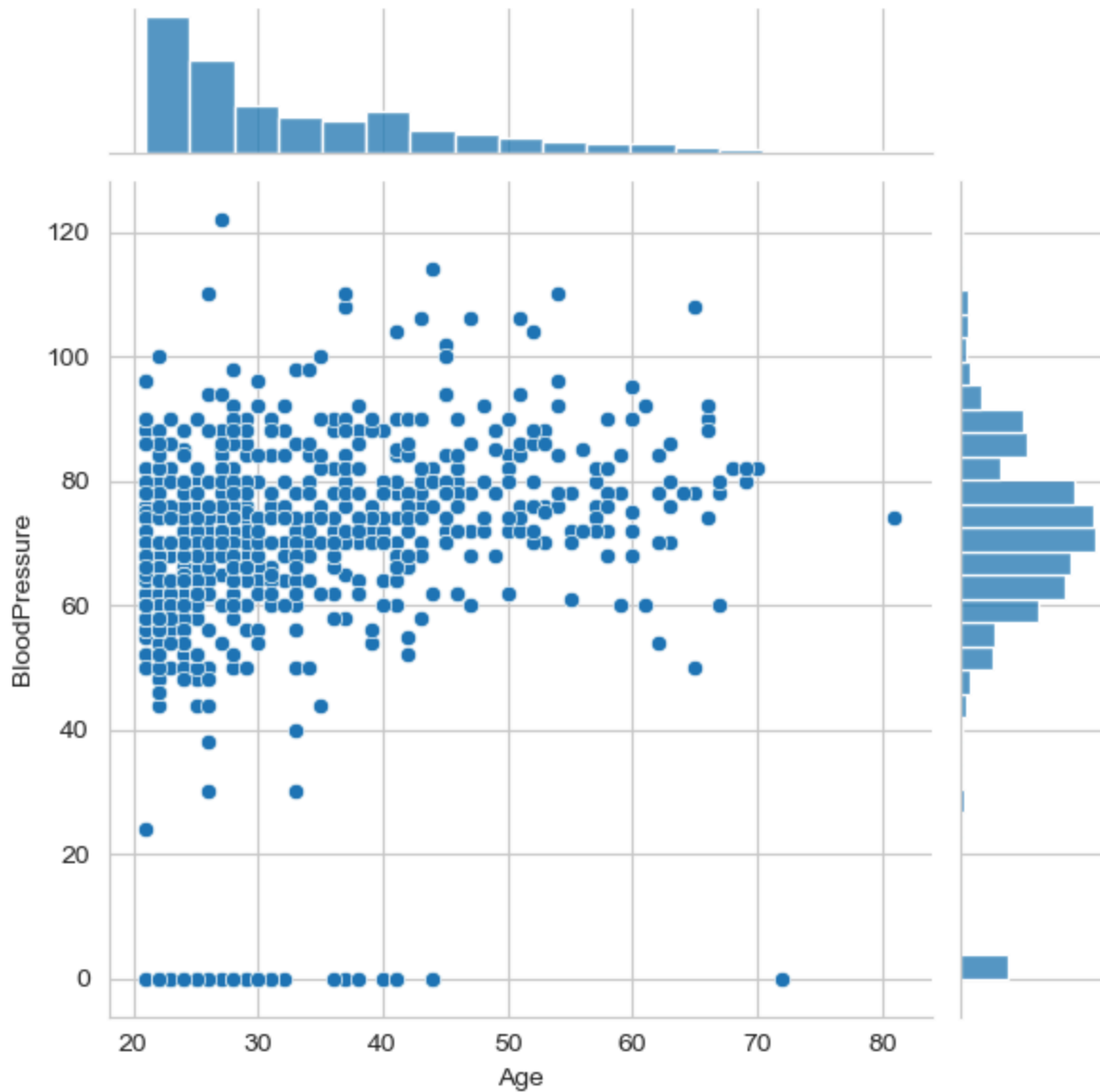
```
sns.distplot(df1['BloodPressure'],kde=False,color='royalblue',bins=20)
```

```
Out[14]: <Axes: xlabel='BloodPressure'>
```



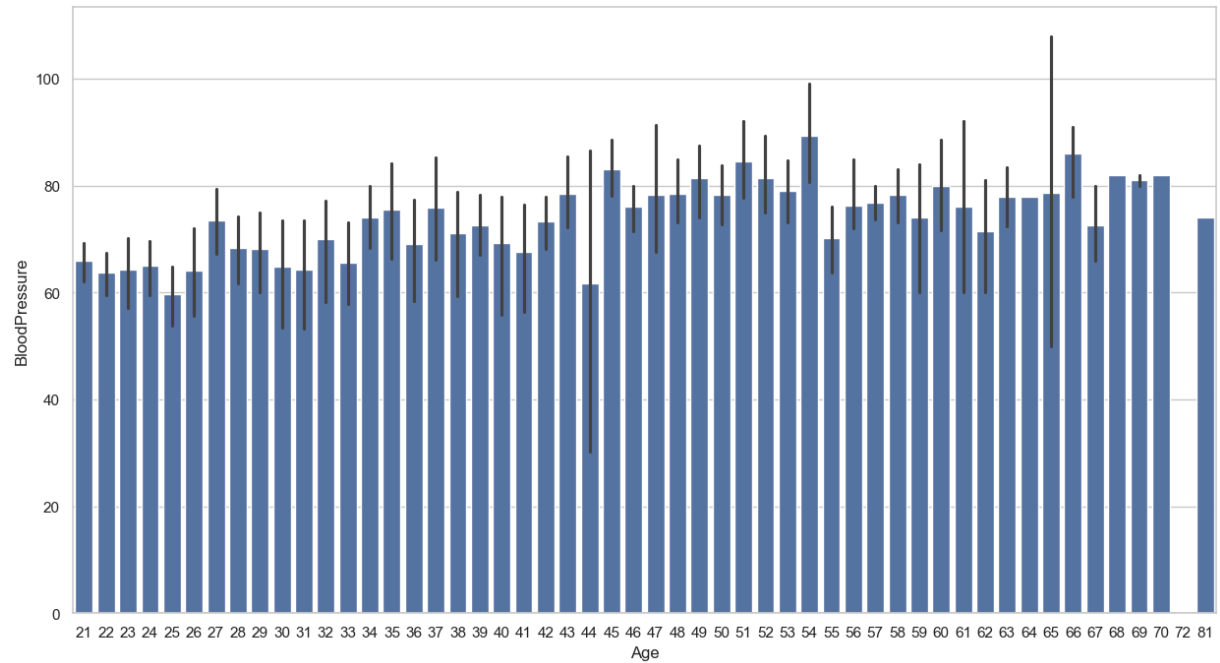
```
In [15]: sns.jointplot(x='Age',y='BloodPressure',data=df1)
```

```
Out[15]: <seaborn.axisgrid.JointGrid at 0x1f13b457a10>
```



```
In [16]: import seaborn as sns
sns.set(style="whitegrid")
#tips=sns.load_dataset("diabetes.csv")
plt.figure(figsize=(15,8))

ax=sns.barplot(x="Age", y="BloodPressure", data=df1,)
```



```
In [18]: from sklearn.model_selection import train_test_split
```

```
In [19]: df1.head()
```

Out[19]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunc
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

◀ ◻ ▶

```
In [20]: x=['Pregnancies','Glucose','BloodPressure','SkinThickness','Insulin','BMI','DiabetesPedigreeFunc']
```

```
In [21]: y=['Output']
```

```
In [22]: df2=pd.DataFrame(data=df1)
df2.head()
```

Out[22]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunc
0	6	148	72	35	0	33.6	0
1	1	85	66	29	0	26.6	0
2	8	183	64	0	0	23.3	0
3	1	89	66	23	94	28.1	0
4	0	137	40	35	168	43.1	2

In [23]:

```
X_train,X_test,y_train,y_test=train_test_split(df1.drop('Outcome',axis=1),df1['Outcome'],
```

In [24]:

```
X_test.head()
```

Out[24]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFu
766	1	126	60	0	0	30.1	
748	3	187	70	22	200	36.4	
42	7	106	92	18	0	22.7	
485	0	135	68	42	250	42.3	
543	4	84	90	23	56	39.5	

In [25]:

```
from sklearn.linear_model import LogisticRegression
LRModel=LogisticRegression(solver='lbfgs', max_iter=7600)
LRModel.fit(X_train,y_train)
```

Out[25]:

LogisticRegression ⓘ ?

Parameters

In [26]:

```
predictions_diabetes=LRModel.predict(X_test)
```

In [27]:

```
from sklearn.metrics import classification_report, confusion_matrix
print(classification_report(y_test,predictions_diabetes))
```

	precision	recall	f1-score	support
0	0.83	0.86	0.85	103
1	0.70	0.65	0.67	51
accuracy			0.79	154
macro avg	0.77	0.76	0.76	154
weighted avg	0.79	0.79	0.79	154

```
In [28]: x=['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunc']
data=[0,170,126,60,35,30.1,0.649,78]
paitentid_54=pd.DataFrame([data],columns=x)
paitentid_54.head()
```

```
Out[28]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunc
0	0	170	126	60	35	30.1	



```
In [29]: df1.head()
```

```
Out[29]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunc
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	



```
In [30]: predictions_diabetes=LRModel.predict(paitentid_54)
```

```
In [31]: print(predictions_diabetes)
```

```
[1]
```

```
In [ ]:
```